Deployment Architecture

Status: proposed. This describes how the system should be deployed to staging and production. The local Docker Compose setup in the README is for development only.

Staging and production design

Overview

The application has three runtime parts: a React single-page app, a Spring Boot API, and a PostgreSQL database. All browser traffic goes through one HTTPS address:

- https://<domain>/ serves the compiled React app (static files).
- https://<domain>/api/v1/* is forwarded to the Spring Boot API.

Serving both from the same origin means the browser never makes cross-origin calls, which keeps CORS simple and lets the API stay private behind the proxy.

The API is stateless (JWT login with SessionCreationPolicy.STATELESS), so it can run as several identical replicas behind the proxy without sticky sessions.

Runtime architecture

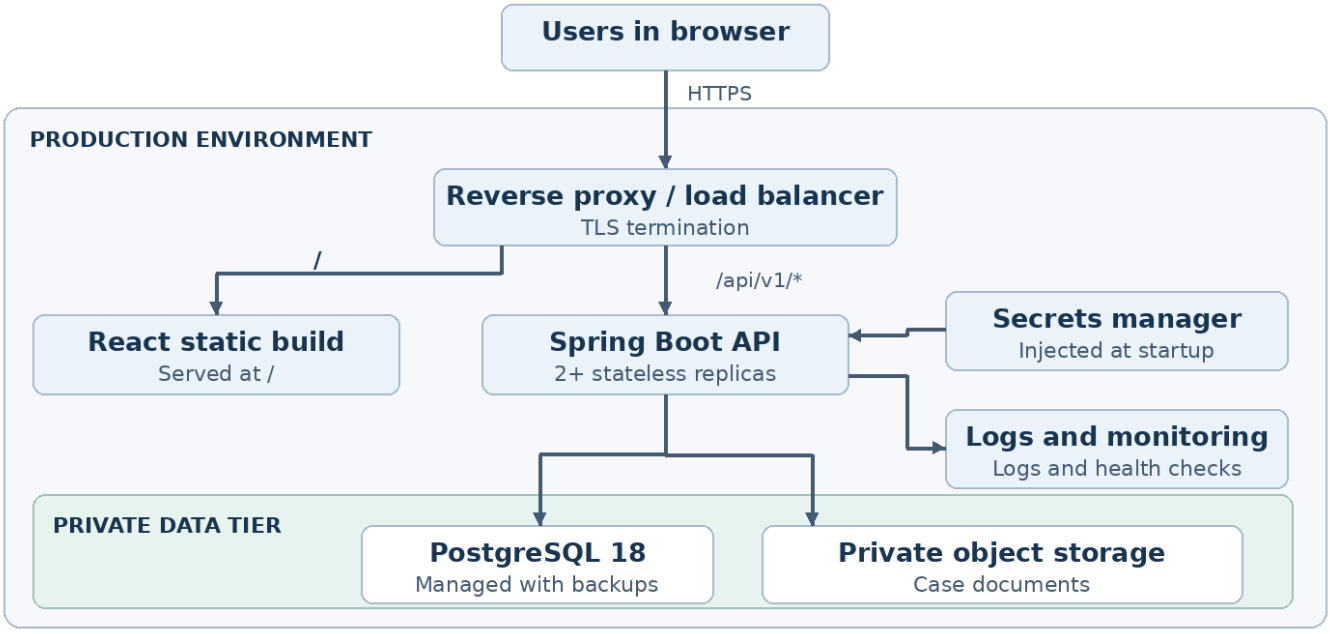


Figure 1. Production runtime and private data services.

Components

Component	Technology	Role	Scaling
Reverse proxy	Nginx, Caddy, or a cloud load balancer	Terminates TLS, serves /, routes /api/v1/* to the API, adds security headers	One managed instance, or two VMs for high availability
Frontend	React 19 + Vite production build (npm run build → dist/)	Static HTML/JS/CSS; no server-side code	Served by the proxy or a CDN; scales for free
Backend API	Spring Boot 4.1 on Java 21 (JRE image from backend/Dockerfile)	Business logic, authentication, Flyway migrations at startup	Stateless; run 2+ containers for zero-downtime deploys
Database	PostgreSQL 18 (managed service recommended)	Cases, clients, hearings, diary notes, users, audit history	Vertical scaling; add a read replica only if needed
Document storage	S3-compatible object storage (private bucket)	Files referenced by documents.storage_path	Effectively unlimited
Secrets manager	Cloud secrets manager, or protected environment files on a VM	DB passwords, JWT secret	–
Monitoring	Cloud logging, or Grafana/Loki/Uptime Kuma on a VM	Container logs, uptime checks, alerts	–

Database logins

Keep the two-login model from the Compose setup:

- **diary_admin** owns the schema. Flyway uses it when the backend starts to apply migrations (DB_MIGRATION_USERNAME / DB_MIGRATION_PASSWORD).
- **diary_app** is used for normal requests (DB_USERNAME / DB_PASSWORD). It can read and write data but cannot change the schema.

Apply docker/postgres/01-create-app-user.sh and 02-app-permissions.sql (or their equivalent) once when you create the managed database.

Network and security

Legal case and client data is confidential, so the layout keeps everything except the proxy off the public internet.

- **Public:** only the reverse proxy, on port 443 (port 80 redirects to HTTPS).
- **Private:** the API containers accept traffic only from the proxy. The database and object storage accept traffic only from the API.
- **TLS everywhere:** HTTPS for users; TLS for API→database connections (sslmode=require in DB_URL).
- **Encryption at rest:** turn it on for the database, its backups, and the document bucket.
- **Documents:** the bucket is never public. The API checks the user's permission, then streams the file or returns a short-lived signed URL.
- **Security headers** on the proxy: Strict-Transport-Security, Content-Security-Policy, X-Content-Type-Options: nosniff, Referrer-Policy: same-origin.
- **Rate limiting** on POST /api/v1/auth/login at the proxy, to slow down password guessing.
- **Admin access:** no public SSH or database ports. Use the cloud console, a bastion host, or a VPN.

Configuration and secrets

The backend is already configured through environment variables. Production values:

Variable	Production value	Secret?
DB_URL	jdbc:postgresql://<private-db-host>:5432/legal_diary?sslmode=require	No
DB_USERNAME / DB_PASSWORD	diary_app and its password	Password: yes
DB_MIGRATION_USERNAME / DB_MIGRATION_PASSWORD	diary_admin and its password	Password: yes
JWT_SECRET	Random, 32+ characters, different in each environment	Yes
JWT_EXPIRATION_MINUTES	60, or shorter if required	No
APP_CORS_ALLOWED_ORIGIN	https://<domain>	No

Secrets are stored in the secrets manager and injected into the containers at start. They are never baked into images, committed to Git, or stored in CI logs. Rotating JWT_SECRET signs everyone out, which is acceptable for an emergency rotation.

CI/CD pipeline

The existing GitHub Actions workflow (.github/workflows/ci.yml) already runs backend tests against PostgreSQL and lints and builds the frontend. Deployment extends it:

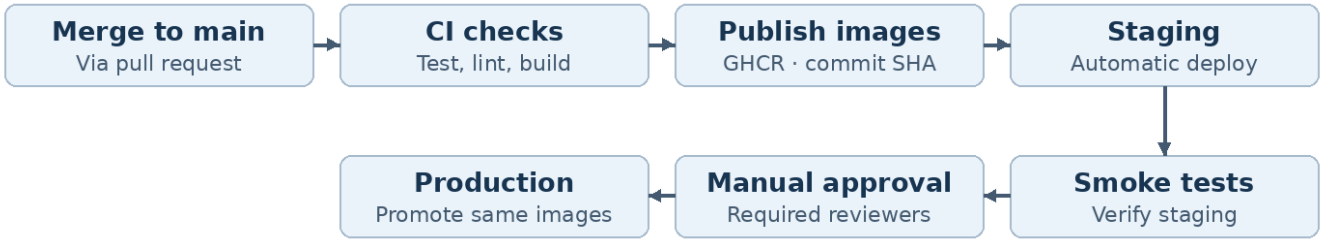


Figure 2. Release workflow from pull request to production.

1. **Build once.** Build the backend and production frontend images once per commit and tag them with the Git SHA. Push to GitHub Container Registry (GHCR).
2. **Staging.** Deploy those exact images automatically, then run smoke tests: the login endpoint responds, and the frontend loads.
3. **Production.** Promote the same image tags after manual approval in a GitHub Environment that has required reviewers.
4. **Rolling update.** Start new API containers, wait until their health check passes, then stop the old ones.
5. **Rollback.** Redeploy the previous image tag. Flyway migrations only move forward, so database changes must stay compatible with the previous app version (add columns first, remove them in a later release).

Environments

Environment	Purpose	How it runs	Data
Local	Development	compose.yaml + compose.dev.yaml (hot reload)	Disposable
CI	Automated tests	GitHub Actions with a PostgreSQL service	Disposable
Staging	Pre-release checks and demos	Same architecture as production, smaller sizes	Fake or anonymised data only
Production	Real users	This document	Real, confidential data

Never copy production data into staging without anonymising it first.

Hosting options

The architecture works on any provider. Choose based on budget and how much you want to operate yourself.

Option A Single Server

One Linux VM running Docker Compose with:

- Caddy as the reverse proxy (automatic Let's Encrypt certificates), serving the frontend dist/ and proxying /api/v1/*.
- The backend container. A second replica on the same VM gives zero-downtime deploys.
- A managed PostgreSQL database. Self-hosted PostgreSQL in a container is possible if you automate nightly off-server backups.

This is simple and cheap, but the server is a single point of failure.

Option B Managed Containers

Component	AWS	Azure	Google Cloud
Proxy / load balancer	Application Load Balancer + ACM	Front Door or Application Gateway	Cloud Load Balancing
Frontend	S3 + CloudFront	Static Web Apps or Blob + Front Door	Cloud Storage + Cloud CDN
Backend	ECS Fargate	Azure Container Apps	Cloud Run
Database	RDS for PostgreSQL	Azure Database for PostgreSQL – Flexible Server	Cloud SQL for PostgreSQL
Documents	S3 (private)	Blob Storage (private)	Cloud Storage (private)
Secrets	Secrets Manager	Key Vault	Secret Manager
Monitoring	CloudWatch	Azure Monitor	Cloud Logging and Monitoring

Backups and recovery

- **Database:** automated daily backups with point-in-time recovery, kept for at least 30 days or longer if legal retention rules require it. Practise a restore into staging at least once a quarter.
- **Documents:** turn on bucket versioning and keep a copy in a second region or account.
- **Targets (suggested):** lose at most 15 minutes of data (RPO) and recover within 4 hours (RTO). Adjust these to the firm's needs.

Work needed before the first deployment

These are gaps in the current code, not in the design:

1. **Production frontend image.** frontend/Dockerfile runs the Vite dev server. Add a multi-stage build that runs npm run build and serves dist/ with Nginx or Caddy, with fallback to index.html for client-side routes.
2. **API base URL.** The frontend should call relative paths (/api/v1/...) so the same build works on every environment. Use a Vite dev-server proxy for local development.
3. **Health endpoint.** Add spring-boot-starter-actuator and expose only /actuator/health (liveness and readiness) for the load balancer and rolling deploys. Today the Compose health check only tests that the port is open.
4. **Document storage service.** Implement file upload and download against object storage, with permission checks, and store the object key in storage_path.
5. **Production profile.** Add application-prod.properties that turns off spring.jpa.show-sql, sets JSON logging, and trusts X-Forwarded-* headers from the proxy (server.forward-headers-strategy=framework).
6. **Release workflow.** Add a GitHub Actions workflow that builds and pushes the images and deploys to staging and production, as described in CI/CD pipeline.